Cover page

Field	Value
Team name	Algo Duo (must match both PDF filenames)
Submission document	`AlgoDuo_FilsPay_Submission.pdf`
Slide deck	`AlgoDuo_FilsPay_Slides.pdf`

Team members and ownership

The repository contains no team roster. The names in `docs/` are scenario characters, so none of them appear here as team members. Fill this table from the actual team.

Member	Owned
Ali Hussain	Orchestrator; API core + settlement + deployment lanes; docs and submission
Ali AlNasheet	Web and admin panels; UI/UX fixes; bug write-ups; demo presentation

Live URLs

All four must be open from outside the demo network at submission time. Replace each placeholder after a `curl -i` returns 200.

Surface	URL
Checkout, public and cookie-free	`http://139.59.44.22/pay/<token>` (generated per payment link; public, cookie-free)
Merchant dashboard	`http://139.59.44.22/` (root is the merchant dashboard)
Admin console	`http://139.59.44.22/admin/`
Public API base	`http://139.59.44.22/v1`

No live public URL exists in the repository. Every host in the docs is a local or development binding (localhost:3000, localhost:5173, localhost:5174, port 8091, port 5432) or an RFC-reserved `.example` placeholder. The four URLs above are runtime facts and stay empty until the deployment is checked. [v: repo-wide search, 2026-10-30]

Repository

Field	Value
Public fork	`https://github.com/ali-afk/fils-pay` (assumed fork; confirm at freeze)
Last commit before the deadline	TODO-PLACEHOLDER: 40-character SHA, plus the short ID GitHub shows
Commit at drafting	`7fd7a959a45715f9334be55c398d5aa5d5bd7c2d` (2026-10-30) [v: `git log -1`]

The fork must contain the AI setup files: `AGENTS.md`, `CLAUDE.md`, `.cursorrules`, the `.omo/` planning workspace, `package.json` scripts, and `scripts/`. Section 6.1 lists them.

---

1. Summary

We found a repository whose documentation, comments, and tests described a finished product that the code did not contain. The refund route returns `501` while the docs say refunds ship. The settlement summary double-counts split captures. Payouts are computed through `float32`, so large amounts drift by a fils. Payment links leak an API-key prefix and accept payment after expiry. Every one of these has a comment nearby that says the behaviour is intentional or already fixed.

We fixed eight defects in the working tree. A public reporting endpoint lacked authentication. A fresh database could not migrate past `0054` or apply `0055`. The dashboard crashed on any negative amount, which includes every refund and every reconciliation gap. The transaction DTO omitted `refundedFils`. Bulk approval could force-approve a rejected merchant. An admin batch date could land two days back near midnight. Settlement status chips and the adjustment notice had no styling, no responsive rules, and no focus ring. The suite stayed at its pre-existing baseline: 418 pass, 16 fail, with no new failures. [lane: 2026-10-30, catalog baseline]

We left the frozen money package and its Go mirror alone, left the reserved `v1` 501 stubs in place, and did not add `refund.succeeded` to the webhook allow-list before the v1.9 release. We also left the integer money columns and the in-memory CSV export as designed, with the open findings documented instead of patched under time pressure.

The main root cause is a broken contract about money. Integer fils is the design (`packages/money/src/fils.ts:1-9`), but the AI convention files say cents (`CLAUDE.md:22-24`, `.cursorrules:5-7`) and the Go payout path computes through `float32` (`payable.go:11`). The same mismatch explains the payout drift; the link to the 10x CSV import is the working hypothesis. [v: `payable.go`; claim: CSV causality]

---

2. Architecture and deployment

2.1 Services, ports, and internet-facing edges

Verified from `ops/docker-compose.yml` on 2026-10-30. The tree changed during the hackathon, so re-check this diagram against the fork at freeze.



Service	Host port	Container port	Faces the internet
api	`127.0.0.1:3000`	3000	Only through the host reverse proxy [claim]
postgres	`5432`	5432	No, but the publish is not loopback-scoped [v: compose:24-25]
settlement	none	8091, internal only	No
reporting	none	none	No
migrate, seed	none	none	One-off jobs

Port facts: api publish is loopback-only (`ops/docker-compose.yml:82`), settlement defaults to internal port 8091 (`apps/settlement/internal/config/config.go:97`), and Postgres publishes `5432:5432` (`ops/docker-compose.yml:24-25`). [v]

Static serving of the two SPAs is not in the tree. There is no `@fastify/static` dependency and no `sendFile` call, so `/` and `/admin` are served by whatever the host proxy does, not by the API. [v: `grep` over `apps/api/src/index.ts`, `apps/api/package.json`, root `package.json`]

2.2 The one command to redeploy

From `ops/README`:

```
docker compose -f ops/docker-compose.yml up -d postgres
docker compose -f ops/docker-compose.yml run --rm migrate
docker compose -f ops/docker-compose.yml run --rm seed
docker compose -f ops/docker-compose.yml up -d api settlement reporting
```

`migrate` and `seed` are one-off containers that exit after the job. Both wait for the Postgres health check. [v: compose `depends_on: service_healthy`]

2.3 Where secrets come from

Secret	Source	Notes
`DATABASE_URL`	`environment:` block in every service	Literal dev credentials in compose: `postgres://filspay:filspay_dev_password@postgres:5432/filspay?sslmode=disable` [v]
`SETTLEMENT_INTERNAL_URL`	api `environment:`	`http://settlement:8091` [v: compose:94]
`SETTLEMENT_INTERNAL_TOKEN`	Must be supplied by the operator. It is **not** in `ops/.env.docker`.	The daemon refuses to start without it (`RequireInternalToken`, `main.go:81`), and the API also refuses in production. With `NODE_ENV=production` from `ops/.env.docker`, a plain compose up fails. The compose comment claiming a `dev-token` fallback is contradicted by the code. [v: executed `loadConfig`, which threw; `main.go:81`]
`WEBHOOK_SECRET_ENCRYPTION_KEY`	Must be supplied by the operator. It is **not** in `ops/.env.docker`.	Read through `env_file`. In production, a missing or malformed key fails startup with an exact message. [v: `config.ts:157-175`; executed `loadConfig`]

Secret	Source	Notes
`ACQUIRER_CALLBACK_SIGNING_SECRET`	`ops/.env.docker`	Sandbox secret, committed to git. `ops/.env.docker` is tracked and not ignored. [v: `git ls-files`; `git check-ignore` returns nothing]
`NODE_ENV`, `PORT`	`ops/.env.docker`	`NODE_ENV=production`, `PORT=3000`

Three fixes before the next deploy. Add `SETTLEMENT_INTERNAL_TOKEN` and `WEBHOOK_SECRET_ENCRYPTION_KEY` to the deployment environment, because the committed `ops/.env.docker` has neither and the stack cannot boot without them. Rotate or remove the committed sandbox acquirer secret before the fork turns public. Give the API its own database role instead of sharing the `filspay` superuser, as the compose TODO says. [v: `ops/.env.docker`; `loadConfig` run; compose:88-90]

2.4 Health checks and restart policies

Check	Behaviour
`GET /healthz`	Process liveness only. Never touches the database. `routes/health.ts:13`. [v]
`GET /readyz`	Boot-only. Reports whether the process connected at startup and never re-checks. A database outage after boot is invisible. The in-file comments claiming otherwise are false. `routes/health.ts:15-21`. [v]
Postgres health	`pg_isready -U filspay -d filspay`, interval 2s, timeout 3s, retries 20. [v: compose:28-32]
Restart policy	`restart: unless-stopped` on postgres, api, settlement, reporting. [v]

Use a real query for live database health. Do not treat `/readyz` as a database monitor.

2.5 Redeploy without interrupting a settlement run

The business day is midnight to midnight in Asia/Bahrain (UTC+3, no daylight saving). The daemon closes the previous day after midnight plus `SETTLEMENT_DAEMON_RUN_DELAY_MINUTES`, default 15. On SIGTERM or SIGINT the context is cancelled, and each batch operation is one Postgres transaction, so a restart commits a whole batch or rolls it back. [v: settlement daemon and batch code]

Restart catch-up handles only the single most recent completed day. Older days need `--date=`. `run-batch --date` is idempotent: it takes a `FOR UPDATE` lock, uses `ON CONFLICT (merchant_id, batch_date) DO NOTHING`, and treats `AlreadyClosed` and `AlreadyPaid` as no-ops. The webhook dispatcher has a 5 second poll fallback beside `LISTEN`, so a dropped listener self-heals without an API restart. [v: `apps/settlement/internal/settlement`, `apps/api/src/webhooks`]

Unresolved: the public ingress path is not in the repo. The compose comment names Caddy on the host, but there is no proxy config to verify. The reporting catch-up path has no row lock and assumes one consumer. [claim; v: `apps/reporting/src/consumer/cursor.ts:23-38`]

Sources: `ops/docker-compose.yml`, `apps/api/src/routes/health.ts`, `apps/api/src/config.ts`, `apps/settlement/internal/config/config.go`, `apps/settlement/internal/settlement/`, `apps/reporting/src/consumer/`, `ops/.env.docker`.

---

3. Capacity

3.1 The fixed scenario

The scenario asks for a working FilsPay: shops get paid, everything is in Bahraini Dinar, 1 BD = 1000 fils, and the app must be built out and hosted. It also asks for a bug note per issue and a security and money audit with the smallest working fix. The done condition includes money right down to the fils, fees and payouts right, the shop receiving the right webhook, and the result visible in the timeline, reports, and reconciliation. [lane: 2026-10-30, OCR of `docs/CASE_FILES.pdf`; OCR noise exists, rendered pages belong in the Appendix]

3.2 Estimate of simulated users

Traffic class	Estimate	Basis
Mixed API traffic	30 to 60 requests per second	Measured 34.1 req/s sustained at 28 VUs with zero errors [v: FP-209]
Interactive dashboard users	30 to 60	One request per action, cached lists, no websockets
Poll-only users	50 to 100	Overview polls one request per 60 seconds per tab [lane: 2026-10-30]
Concurrent heavy CSV exports	3 to 4	p50 2.21 s, about 20 MB per export, peak RSS 236 MB of 512 MB [v: FP-209]

Over days, not demo minutes, the binding limit is unbounded `outbox_events` growth. [claim: db-query-retention plan; growth mechanics verified]

3.3 Load test: tool, method, results

Source: `docs/perf/fp-209-loadtest-2026-09.md`. The run is from September, not from this deployment, so re-run it against the fork and save the raw JSON before submission. The original raw JSON is missing from the repo. [v: file read; missing artifact checked]

Item	Value
Tool	k6 0.52, 10 minute run, three scenarios

Item	Value
API limits	`--cpus=0.5 --memory=512m`, Node 20 slim, separate Postgres 16 container
Dataset	Delta Wholesale snapshot, 18,432 transactions, 2 July to 1 September 2026
VUs	export_csv 3, list_transactions 20, settlement_summary 5
Total throughput	20,471 requests over 600 s, about 34.1 req/s

Scenario	Iterations	p50	p95	Max	Errors
export_csv	141	2.21 s	4.18 s	5.93 s	0
list_transactions	18,120	41 ms	112 ms	488 ms	0
settlement_summary	2,210	88 ms	240 ms	701 ms	0

Memory metric	Value
RSS at idle	171 MB
RSS peak	236 MB
Heap peak	118 MB
OOM kills	0
Restarts	0

Secondary evidence: raising Postgres `shared_buffers` from 32 MB to 128 MB cut the settlement summary query from 3.4 s to 410 ms warm, with no code change (`docs/perf/2026-07-settlement-summary-slow.md`). [v]

3.4 Bottleneck

The API database pool is the first limit: `pg.Pool max` defaults to 5 (`apps/api/src/config.ts:206`, `apps/api/src/db/pool.ts:18`), with a 5 second connect timeout. Under the FP-209 mix, latency came from connection queueing, not CPU. Export memory is the second limit, because the CSV path loads the full result set. The settlement summary query is the third: it scans `transaction_events` without an index on `(transaction_id, event_type)`, and that index is still absent. [v: pool and query read; missing index checked]

3.5 How the stack turns away excess load and recovers

Implemented today: pool cap with fail-fast connect timeout, sequential batch dispatch (500 then 100), atomic consumer batches with a watermark in one transaction, `LISTEN` plus a 5 second poll fallback, an 8 MB KYC upload cap, and 30 second idle timeouts. [v]

Not implemented: adaptive connection shedding, a local nginx layer, outbox purge, the `transaction_events` index, and response schemas. The honest summary is deliberate backpressure through a small pool and idempotent batch semantics, with no graceful HTTP shedding or rate limiting yet. [v: absent files and configs checked]

Sources: `docs/perf/fp-209-loadtest-2026-09.md`, `apps/api/src/config.ts`, `apps/api/src/db/pool.ts`, `apps/settlement/internal/settlement/query.ts`, `apps/reporting/src/consumer/`, `docs/CASE_FILES.pdf`.

---

4. Findings report

Status language is fixed by the catalog: `fixed in this tree` means an uncommitted working-tree change, `open` means not fixed here, `planted` means the comment or doc lies and the code is the evidence. The fork may differ at freeze. Re-check every row against the fork.

4.0 Critical bugs (must-read)

Nine items decide whether the money is right and whether the app can be trusted. Fix status is against this working tree.

Critical bug	Evidence	Status
Payouts compute through `float32` and drift above 2^24 fils	`apps/settlement/internal/settlement/payable.go:11`; 16,777,200 fils returns 16,777,199 [v]	OPEN. Fix is `grossFils - feeFils` plus a regression test.
Refunds never settle: the producer is a 501	`apps/api/src/routes/refunds.ts:24`; no refund claim step in CloseBatch [v]	OPEN. This is the missing feature the scenario asks to build.
Refunds fail on `chk_refunds_reason_not_null`	8 refund tests and 3 idempotency tests fail on that constraint [lane: 2026-10-30]	OPEN. Blocks the refund demo and accounts for part of the 16-failure baseline.
`export-summary` was unauthenticated and always 500	`routes/reporting.ts:79` had no `preHandler` [v]	FIXED in this tree. `reportingAuth.test.ts` proves 401 / 200.
Fresh database migrate dies at `0054`, and `0055` can never apply	`db/migrations/0054_views.sql:20-22`; `0055` shape mismatch [v]	FIXED in this tree. Fresh DB migrated 66 files, 0 errors.
Bulk approve force-approves rejected merchants	`apps/api/src/admin/bulkKyc.ts` had no status predicate [v]	FIXED in this tree. `bulkApproveGuard.test.ts` added.
Webhook deliveries table holds leftover test residue	Environmental, database state rather than application code [claim]	ENVIRONMENTAL. Clean the rows before the demo and record what was removed.

Critical bug	Evidence	Status
OTP delivery stub logs nothing in production	`apps/api/src/auth/notifier.ts`; the stub never delivers and logs no code [lane: 2026-10-30]	OPEN. The live demo needs a real delivery path or a documented bypass.
Webhook signature enforcement exists, but the simulator never signs	`ACQUIRER_CALLBACK_REQUIRE_SIGNATURE` at `config.ts:259`; simulator does not sign [v: config; lane: simulator]	OPEN. Flipping enforcement changes live behaviour with no test or demo benefit.

Read the rows marked OPEN as open risks, not as shipped work. The two money-path items (float32 and refunds) are the highest-impact defects in the repository.

4.1 Fixed in this working tree

ID	Symptom and how it was found	Root cause	Fix	Verified by
B1 (R16/G4)	`GET /v1/reporting/export-summary` returned 500 to anonymous callers and worked for merchants. Found by route audit.	The route had no `preHandler` while its three siblings did, then dereferenced `request.merchantAuth!.merchantId`. `routes/reporting.ts:79`.	Added `{ preHandler: requireMerchantAuth }`.	New `reportingAuth.t` 401 anonymous, 200 authenticated; suite a clean.
B2 (C5)	Fresh database migrate died at `0054_views.sql` with `42P01: relation "settlement_batch_events" does not exist`.	The view referenced a table that exists in no migration. `db/migrations/0054_views.sql:20-22`.	Replaced the subquery with the real eligibility rule: `te.settlement_batch_id IS NULL`.	Fresh DB `filspay_` migrated all 66 files, view queried.
B3 (C6)	Migration `0055` could never apply on a fresh DB.	It inserted columns and a `suspense` kind that `0007` does not define.	Re-shaped to `INSERT (id,name,kind)` with `asset`, using the repo's own precedent.	Fresh-DB migrate cle
B4 (W1)	Dashboard crashed on any refund event or negative reconciliation gap with a blank route.	`bhd()` called `toFils` without `allowNegative`, and `toFils` throws on negatives. `apps/web/src/format.ts:11`; `packages/money/src/fils.ts:49-51`.	Added `bhdSigned()` and used it on the timeline and both gap displays.	Web typecheck and b suite attribution-clear
B5 (G6)	Clients could not compute the refundable balance; a comment claimed the field was there.	`refundedFils` was selected by `lookup.ts:21` but never serialized. `transactions/serialize.ts`; `types.ts:89`.	Added it to the row type, DTO, serializer, and web type.	Typecheck and build row sources use `SEl`
B6 (A1)	Admin bulk-approve could force-approve a rejected merchant.	The update had no status predicate despite a JSDoc promise. `admin/bulkKyc.ts`.	Added `AND onboarding_status = 'under_review'`.	Existing test green; n `bulkApproveGuar` proves a rejected id s rejected.
B7 (A3)	Admin "Run batch" default date could be two days back at 00:30 Bahrain time.	`yesterdayIso()` used UTC `toISOString()` while the business day is UTC+3. `apps/admin/src/pages/settlement/SettlementPage.tsx:18-23`.	Format from local date components.	Typecheck and build
B8 (W9/A9)	Settlement statuses were uncoloured, the adjustment notice unstyled, no responsive layout, no focus ring.	Bare status text and missing CSS.	Status chip, notice class, responsive and `:focus-visible` blocks, `partially_refunded` colour.	Web and admin type build green.

4.2 Open findings

ID	Symptom and reproduction	Root cause	Fix not yet applied	Verified by	Effect on existing data
F1	Payouts above 16,777,216 fils drift. A 16,777,200 fils figure returns 16,777,199.	`float32(gross)/1000 - float32(fee)/1000` then round. `payable.go:11`.	`grossFils - feeFils`, plus a locked regression test above 2^24.	Read of `payable.go`; arithmetic reproduced. [v]	Rows above 2^24 fils only. Repair through batch recalculation after the fix.
F2	Refunds never reach settlement, payouts, or reconciliation. `POST /v1/refunds` returns 501.	Producer is a stub; `settlementClient.ts:292` calls a non-existent `/internal/refund-adjustment`; CloseBatch has no refund claim step. `routes/refunds.ts:24`.	Build the refund lane end to end, then add the settlement adjustment.	Read of route and client. [v]	None today: no refund can succeed. This is the missing feature in the scenario.
F3	A 60k plus 40k split capture reads as 200k fils in the settlement summary.	The CTE joins transactions to capture events and sums `captured_fils`, so multi-capture rows duplicate. `apps/api/src/settlement/query.ts:368`.	Deduplicate by transaction and take each capture once.	Read of CTE. [lane: 2026-10-30]	Summary totals overstate split captures.
F4	Reconciliation shows a negative gap.	The query subtracts payouts without removing the payout debit.	Rework the reconciliation query to net the debit once.	[lane: 2026-10-30]	Reconciliation report only.

ID	Symptom and reproduction	Root cause	Fix not yet applied	Verified by	Effect on existing data
F5	Payment-link captures do not apply the discount; direct payments do.	Discount logic sits after the first commit on the link path.	Move discount application into the same transaction.	[lane: 2026-10-30]	Link-paid transactions overcharge the merchant.
F6	Discount adjustment uses only the latest capture event on a split capture.	Adjustment query picks one event.	Apply the adjustment to the full capture set.	[lane: 2026-10-30]	Adjustment totals wrong for split captures.
F7	Public payment-link read returns the merchant API-key prefix and key hash.	The link payload serializes key metadata.	Return a public-safe payload.	[lane: 2026-10-30]	Credential metadata exposed to anyone with the link. Rotate any exposed key.
F8	A link that displays as expired still accepts payment.	No server-side expiry or auth gate on checkout.	Enforce expiry and state on the server.	[lane: 2026-10-30]; support-ticket exhibit	Post-expiry payments exist. Refund or reconcile them manually.
F9	Merchant OTP can be issued to another merchant's user with the same email.	OTP lookup is not scoped by merchant. `routes/auth.ts`.	Scope the lookup by merchant and user.	[lane: 2026-10-30]	Potential cross-tenant login. Audit OTP issuance logs.
F10	A crash or finalize failure in the payment-link flow can charge twice.	The flow is not one transaction.	One transaction with an idempotency key.	[lane: 2026-10-30]; ticket "link charged twice"	Duplicate charges exist. Reconcile by acquirer reference.
F11	"Recompute transaction statuses" can clear a test merchant's `test_mode`.	The admin tool overwrites the flag.	Preserve `test_mode` in the update.	[lane: 2026-10-30]	Test merchants may be marked live. Re-set the flag.
F12	Webhook dispatcher can duplicate deliveries with more than one API replica.	`LISTEN` race across replicas.	Single-consumer claim or advisory lock.	[lane: 2026-10-30]; duplicate payout ticket	Duplicate webhook deliveries. Receivers must be idempotent.

4.3 Root causes

Money through floating point and a wrong unit contract. F1 and the 2-decimal assumptions in the AI configs share one cause: ADR-0001 says integer fils, and parts of the system use cents or floats. The 10x CSV import fits the same fingerprint, though the causal link is not proven line by line. [v: `packages/money/src/fils.ts:1-9`; `.cursorrules:5-7`; `CLAUDE.md:22-24`; claim: CSV causality]

Test-locked wrong behaviour and planted comments. Several findings sit beside comments that assert the opposite of the code: `format.ts:9-10` says amounts are never negative, `docs/ADMIN.md:31-32` says only `under_review` merchants are affected, `db/migrate.ts:67` claims a checksum that does not exist. The repository's own story says tests were locked to the wrong behaviour on purpose. Treat that story as the repo's interpretation, not as a team claim; the technical defect is verifiable either way. [v: the listed lines; `claim` for the narrative]

Incomplete feature wiring. Refunds are 501, the SPA has no `/` route, and CORS is absent. The docs say these shipped. They did not ship in this tree. [v]

Atomicity gaps. F5, F10, and F12 apply single-transaction discipline in some paths and not others. The fix is the same shape each time: one transaction or one idempotent claim.

Query design and missing indexes. F3, F4, F6, and the slow summary share join and index problems, not business-logic problems.

4.4 Decisions and open risks

Decision	Why	Safeguard we propose	Status
Left `packages/money/` and `apps/settlement/internal/money/` untouched	Frozen after the Q2 audit; rounding matches the acquirer's test vectors	Any change needs finance approval and re-certification	Closed for this submission
Left `v1` 501 stubs in place	Reserved for upcoming partner features	Track them as stubs; do not remove	Open by design
Did not add `refund.succeeded` to `WEBHOOK_ELIGIBLE_EVENT_TYPES`	That event ships in v1.9; adding it early stalls the queue because `envelope.ts` has no case	Add the envelope case first, then the event type	Open
Did not widen integer money columns	BHD in fils fits in 32-bit; widening needs a DBA migration plan	Migrate to BIGINT in one reviewed migration	Open risk at very large volume
Kept the in-memory CSV export	Date windows are capped, so result sets stay small	Stream only with resumable downloads	Open risk above 100k rows
Did not fix F1 to F12 in this lane	Each needs a locked regression test and a money-path review	Run them as separate fixes with tests; re-verify at freeze	Open
Committed sandbox secret in `ops/.env.docker`	The file is tracked and the fork will be public	Rotate `ACQUIRER_CALLBACK_SIGNING_SECRET`, then remove the value	Must fix before submission

Decision	Why	Safeguard we propose	Status
Missing internal token and webhook key in `ops/.env.docker`	The file sets `NODE_ENV=production`, and both services fail startup without those values	Supply both through the deployment environment, never in the committed file	Must fix before the next deploy
`/readyz` is boot-only	Documented base-platform behaviour	Monitor with a real query from outside	Open
Reporting catch-up has no row lock	Assumes one consumer	Add `FOR UPDATE` or a single-consumer claim	Open

Sources: `docs/bug-explanations-2026-10-30.md`, `docs/adr/`, `apps/api/src/routes/refunds.ts`, `apps/api/src/webhooks/dispatcher.ts`, `db/migrate.ts`, `ops/.env.docker`.

---

5. UI and UX changes

Twenty-six screenshots already exist under `docs/plans/evidence-webapp-uiux-2026-10-30/`, captured on 2026-10-30 at 21:58. They cover both panels, the checkout at desktop and mobile widths, and the admin console. Use them as the starting set and capture a before and after pair for each changed state where the pair is missing. Keep one browser window at 1440x900 and the light theme. [v: directory listing]

5.1 A dashboard that no longer crashes

What: signed formatting for refund events and reconciliation gaps.

Why: a single refund event or negative gap blanked the page because `toFils` throws on negatives. This made refunds invisible exactly when a merchant needed them.

Proof: `bhdSigned()` in `apps/web/src/format.ts:15-16`, used at `TransactionDetailPage.tsx:242` and `ReconciliationPage.tsx:115,131`. Web typecheck and build green. [v]

Screenshots: TODO-PLACEHOLDER: before, transaction timeline with a refund event crashing. After, same timeline rendering `-4.750 BHD`.

5.2 Refundable balance shown in the transaction

What: `refundedFils` in the API response, serializer, and web type.

Why: the UI could not compute how much was left to refund, and a comment claimed the field already existed.

Proof: `serialize.ts:28,46`; `types.ts:93`. Typecheck and build green. [v]

Screenshots: TODO-PLACEHOLDER: before, no refundable figure. After, transaction detail with the refunded amount.

5.3 Safer bulk approval

What: bulk approve now only touches merchants in `under_review`.

Why: one click could approve a rejected merchant and pass KYC.

Proof: new `bulkApproveGuard.test.ts`; rejected id stays unchanged. [v]

Screenshots: TODO-PLACEHOLDER: before and after of the admin bulk-approve result table, with a rejected row left untouched.

5.4 Correct admin batch date near midnight

What: the "Run batch" default date uses local components instead of UTC.

Why: at 00:30 in Bahrain the old default pointed two days back, risking a batch on the wrong day.

Proof: `SettlementPage.tsx:18-23`; typecheck and build green. [v]

Screenshots: TODO-PLACEHOLDER: before, date field one day early. After, yesterday in Asia/Bahrain.

5.5 Readable settlement states

What: status chips, a styled adjustment notice, responsive layout, and visible focus rings.

Why: statuses were bare text, `partially_refunded` had no colour, and keyboard users could not see focus.

Proof: `SettlementBatchDetailPage.tsx:154`, `AdjustmentImportPage.tsx:140`, new `@media` and `:focus-visible` blocks. Builds green. [v]

Screenshots: TODO-PLACEHOLDER: before and after of a settlement batch with a `partially_refunded` line, plus a keyboard-focus capture.

5.6 Changes that exist only as plans

Two items in the plan are not in this tree and must not be presented as shipped: a dashboard root redirect and a CORS plugin. The router has no `/` route today, so a browser at `/` shows "No routes matched location /". There is no `apps/api/src/plugins/cors.ts`. If the fork ships them by freeze, capture

before and after and move them into the shipped list. [v: `apps/web/src/router.tsx`; `ls apps/api/src/plugins`]

Recording: TODO-PLACEHOLDER: link the demo recording, or write "not recorded" and attach the screenshot set to the Appendix.

---

6. AI usage

6.1 Setup and where it lives

Path	Lines	What it does
`AGENTS.md`	90	Repository conventions: frozen money package, parsing rules, API stubs, database rules. [v: `wc -l`]
`CLAUDE.md`	81	Agent instructions, including the money unit. [v]
`.cursorrules`	72	Cursor rules, including the money unit and settlement timing. [v]
`.omo/`	directory	Planning-lane artifacts: plans, code review, batch commits. [v: `ls .omo`]
`scripts/`	directory	`expire_holds.ts`, `reconcile_march.ts`, `lib/sqlFilter.ts`, plus tests. [v]
`package.json` scripts	n/a	`typecheck`, `test`, `migrate`, `seed`.
`db/migrate.ts`, `db/seed/index.ts`	n/a	Idempotent migration runner and self-verifying seed.

One line for the slide: three root agent-convention files, an `.omo` planning workspace, and scripted tooling, with no CI.

These files are judge-visible and are where the strongest AI story lives. The conventions say cents; the money package and its tests say fils.

6.2 Annotated transcript excerpts

The guidelines require three excerpts from real sessions. Fabricating one is disqualifying. The three below are templates with the exact evidence already verified in the repo. Each has a `TODO-PLACEHOLDER: paste real transcript` block. A transcript must come from the team's own session log, with the prompt, the relevant answer, and the follow-up.

Excerpt 1: the AI convention files said cents, the code said fils

Finding it supports: the money-unit mismatch behind the 10x CSV import and the refundable-balance confusion. This is the required "AI was wrong" excerpt.

Prompt and context: TODO-PLACEHOLDER: paste the prompt that asked the agent to write or review `CLAUDE.md` and `.cursorrules`, and list the tools it had (file read, grep, test run).

Relevant part of the answer: TODO-PLACEHOLDER: paste the lines where the agent states that money is integer cents, 1 BD = 100 cents.

Evidence verified now:

- `.cursorrules:5-7`: "This project uses cents (1 BD = 100 fils)... the actual unit is cents... 2-decimal formatting."
- `CLAUDE.md:22-24`: "integer cents (1 BD = 100 cents)... historical name."
- `packages/money/src/fils.ts:1-9`: BHD is 3-decimal, 1 BD = 1000 fils, integer fils only, never floats or `toFixed(2)`.
- Test evidence: `parseBhdToFils("4.75") === 4750` (`packages/money/src/fils.test.ts:172`), and the executed parser returns 4750. [v: `bun -e` import + call; test file read]

How we noticed: the convention files disagreed with each other and with the money package. A 10x import error is the fingerprint of a 100 versus 1000 assumption. Running the parser test settled it.

What changed: TODO-PLACEHOLDER: state whether the team corrected the convention files, added a lint or test guard, or only recorded the finding.

Status: NEEDS-REAL-TRANSCRIPT. Do not ship this section with the placeholder still in it.

Excerpt 2: "re-running is safe" next to a fresh UUID

Finding it supports: the idempotency lie in the March reconciliation script.

Prompt and context: TODO-PLACEHOLDER: paste the prompt that produced or reviewed `scripts/reconcile_march.ts`, and note whether the agent could run the integration test.

Relevant part of the answer: TODO-PLACEHOLDER: paste the part where the agent claims the script is safe to re-run because `entry_group_id` deduplicates.

Evidence verified now:

- `scripts/reconcile_march.ts:71`: "am: re-running is safe, entry_group_id dedupes it".
- `scripts/reconcile_march.ts:74`: `const entryGroupId = randomUUID();`, a new value on every run.
- The insert has no `ON CONFLICT` on `entry_group_id` and no uniqueness guard in the shown block. [v: lines 71-80 read]

How we noticed: reading the two adjacent lines. A fresh UUID cannot deduplicate anything.

What changed: TODO-PLACEHOLDER: state the fix, for example a stable group key plus a unique index, or that the script is quarantined.

Status: NEEDS-REAL-TRANSCRIPT.

Excerpt 3: the empty commit where the AI could not explain its own code

Finding it supports: the float32 payout bug and the habit of trusting a reassuring comment.

Prompt and context: TODO-PLACEHOLDER: paste the request that asked the agent to explain `settlement/batch.go` or `payable.go`, and list the files it could read.

Relevant part of the answer: TODO-PLACEHOLDER: paste the answer that failed to explain the float32 path, or the moment it repeated "intentional" without evidence.

Evidence verified now:

- Commit `925744c5116f2455048e4262c2f7db329de7a786`, author Yousif Al-Ansari, subject: "doubt: asked claude to explain the code claude wrote. it could not." The commit is empty: `git show --stat` lists no files. [v: `git show`]
- The code it doubted is still there: `apps/settlement/internal/settlement/payable.go:11` computes `float32(grossFils)/1000 - float32(feeFils)/1000`. [v]
- A nearby comment calls the behaviour intentional; the arithmetic says otherwise above 2^24 fils.

How we noticed: the commit message itself records the doubt, and the code still contains the defect. The human read the diff after the AI could not justify it.

What changed: TODO-PLACEHOLDER: state whether `payable.go` was fixed in the fork, and link the regression test. If it is still open, say so.

Status: NEEDS-REAL-TRANSCRIPT.

Appendix

All items below are optional evidence for judges and belong in the PDF after Section 6.

1. Rendered `docs/CASE_FILES.pdf` pages plus the OCR transcript, with the OCR caveat.
2. FP-209 raw k6 JSON and the graph from the submission-day rerun. TODO-PLACEHOLDER: attach the new run, because the original JSON is missing.
3. `go test ./...` tail from `apps/settlement`, with date.
4. `bun test` tail: 418 pass, 16 fail at the current baseline, with the named pre-existing failures. TODO-PLACEHOLDER: attach the submission-day tail.
5. Fresh-database migration log showing 66 files and zero errors.
6. `settlement export-scheme-file --batch=<uuid>` sample.
7. Reconciliation log from `scripts/out/`.
8. `EXPLAIN` before and after for the settlement summary query.
9. The before and after screenshot set from Section 5, starting from `docs/plans/evidence-webapp-uiux-2026-10-30/` (26 files, verified present).
10. The repository inventory for Section 6.1, with `wc -l` output.

Verification tag key

- `[v: path:line]` verified in this working tree on 2026-10-30 by a read or command.
- `[lane: 2026-10-30]` verified by the audit or planning lanes on 2026-10-30, re-check at freeze.
- `[plan]` claimed by a `docs/plans/` document and not re-verified here.
- `[claim]` not verified, or a doc or comment statement that the code contradicts.
- `TODO-PLACEHOLDER` team-specific fact that only the lead can fill.
- `NEEDS-REAL-TRANSCRIPT` must be replaced with a real session excerpt before submission.

Known unverified items

- Live URLs, team names, fork link, and final commit ID.
- The host reverse proxy and its TLS configuration. Compose comments name Caddy; no config is in the repo.
- Whether the fork contains the working-tree fixes. Re-check every status at freeze.
- The exact values of `SETTLEMENT_INTERNAL_TOKEN` and `WEBHOOK_SECRET_ENCRYPTION_KEY` on the live box. The repo does not contain them, and the committed environment cannot boot without them.
- The submission-day load test and test-suite tails.